Recap Introduction to Python Programming

How Are We Going to Do This?

- I don't want to just repeat everything
- I want you to ask all your questions
- I want to give you a different perspective
- I don't want to overload you

So my suggestion

For every topic

- Do we really need it
- Why do we use it
- Pitfalls / common mistakes

Variables

```
In [ ]: booleans: bool = True
        integers: int = 7
floats: float = 7.5
strings: str = "Some Text"
```

We need a way to store data

```
In [1]: import random
        for _ in range(random.randint(1, 4)):
    print('Hey')
for _ in range(random.randint(1, 4)):
    print('Ho')
```

```
Hey
Hey
Hey
Hey
Ho
```

```
In [2]: import random
        the_variable = random.randint(1, 4)
for _ in range(the_variable):
    print('Hey')
for _ in range(the_variable):
    print('Ho')
```

```
Hey
Hey
Hey
Ho
Ho
Ho
```

We wouldn't be able to guarantee the same number of prints without storing the results somewhere.

But do we really need to be able to create variables?

```
In [3]: a_variable = 0 # Creating a Variable

data = [None] * 100 # Solves All Our Data Storing Needs
```

Technically not, one big list already does the trick.

```
In [5]: import random
        data[45] = random.randint(1, 4)
for _ in range(data[45]):
    print('Hey')
for _ in range(data[45]):
    print('Ho')
```

```
Hey
Hey
Ho
Ho
```

This is a real script from a programming language

```
>++++++++[<+++++++++>-]<.>++++[<+++++++>-]<+.+++++++..+++.>>++++++[<+++++++>-]<+
+.------------.>++++++[<++++++++++>-]<+.<.+++.------.--------.>>>++++[<++++++++>-
]<+.
```

It would print 'Hello, World'

My Point

- Most programming concepts are not necessary to achieve your goal.
- They make your life a lot easier though.

```
In [6]: data[0] = "Fast"
        data[1] = 400
data[2] = 0
data[3] = "Furious"
data[4] = 401
data[5] = 0

data[6] = 0
while data[6] < 3:
    data[2] += data[1]
    data[5] += data[4]
    print(f"{data[0]} is at {data[2]}")
    print(f"{data[3]} is at {data[5]}")
    data[6] += 1
```

```
Fast is at 400
Furious is at 401
Fast is at 800
Furious is at 802
Fast is at 1200
Furious is at 1203
```

```
In [7]: name = "Fast"
        speed = 400
position = 0
name2 = "Furious"
speed2 = 401
position2 = 0

clock = 0
while clock < 3:
    position += speed
    position2 += speed2
    print(f"{name} is at {position}")
    print(f"{name2} is at {position2}")
    clock += 1
```

```
Fast is at 400
Furious is at 401
Fast is at 800
Furious is at 802
Fast is at 1200
Furious is at 1203
```

Variables

- Allow you to store and access data
- Allow you to communicate the meaning of your program

Variables - Pitfalls

- Bad Naming
    - `x` and `y` are fine for coordinates
    - Not good to represent a name
    - `for x in names:`
- Consistency

```
In [8]: a = "a crazy string  "
        b = a.replace(' ', '')
c = a.capitalize()
print(c)
```

```
A crazy string
```

In [9]:
```python
# overwriting
names = ["Person I'm looking for", "This other dude"]
for name in names:
    if name == "Person I'm looking for":
        found = True
    else:
        found = False # This will overwrite the final value
print(f"The person is in the list: {found}")


# scopes
my_variable = 5

def my_function():
    my_variable = 6 # Different scope

print(my_variable)
my_function()
print(my_variable) # The variable will not be changed
```

```
The person is in the list: False
5
5
```

Conditionals

```
In [ ]: if 3 > 4:
            print("Impossible!")
elif 1+3 < 12:
    pass
else:
    print('Something')
```

We need a way to make decisions.

```
In [ ]: def print_dog_or_cat(person: Person):
            if person.loves('dogs'):
        print('dog')
    else:
        print('cat')
```

Every time we measure or observe something we want to be able to do different things based on this observation.

But do we need if, elif and else?

```
In [10]: # We can do the others using if

if 3 > 4:
    print("Impossible!")
if not 3 > 4 and 1+3 < 12:
    pass
if not 3 > 4 and not 1+3 < 12:
    print('Something')

# But we could go further than that

def positive(): return "Positive!"
def negative(): return "Negative!"

x = -5
actions = {True: positive, False: negative}
print(actions[x > 0]())
```

```
Negative!
```

Conditionals

- Allow you to branch execution based on conditions.
- `if else` is easy to read and write

Conditionals - Pitfalls

- Repeated ifs, forgetting the else or elif

```
if True:
    print(1)
if False: # start of a new if else block
    print(2)
else:
    print(3) # will be printed
```

In [ ]:
```python
# Be careful with nesting

def categorize_number(num):
    if num > 0:
        if num % 2 == 0:
            return "Positive Even"
        else:
            return "Positive Odd"
    else:
        if num == 0:
            return "Zero"
        else:
            if num % 2 == 0:
                return "Negative Even"
            else:
                return "Negative Odd"

def categorize_number(num):
    if num == 0:
        return "Zero" # Often you can return early

    # You can build up your final results gradually
    sign = "Positive" if num > 0 else "Negative"

    if num % 2 == 0:
        return sign + " Even"
    return sign + " Odd"
```

Iteration

```
In [11]: stuff = [1,2,3]

for item in stuff:
    print(item)

while len(stuff):
    print(stuff[0])
    stuff = stuff[1:]
```

```
1
2
3
1
2
3
```

We want to repeatedly execute the same code

```
In [ ]: # We can just copy paste
        for i in range(3):
    do_something()

do_something()
do_something()
do_something()

# Or we can use functions
def like_a_while_loop(n: int) -> None:
    print(n)
    if n < 5:
        like_a_while_loop(n + 1)
```

Iteration - Pitfalls

- When to stop
- Nested iteration
- Infinite loops
- Forgetting built in functions

```
In [ ]: names = ["Person I'm looking for", "This other dude"]
        for name in names:
    if name == "Person I'm looking for":
        found = True
        break # stopping the loop here will fix the overwriting
    else:
        found = False
print(f"The person is in the list: {found}")

# Nested
for game in tournament:
    for team in game.teams:
        for player in team:
            pass
            # things will get complicated very quick
            # Functions can make this more manageable
```

In [13]:
```
# infinite loops
n = 6
while n > 5:
    print(n)
    # if we do not update n this will go on forever
    break # we can also stop forcefully

# build in functionality
scores = {"Alice": 85, "Bob": 90, "Charlie": 78}

# Calculate the sum of all scores
total_score = sum(scores.values())

# Check if a specific person is in the dictionary
name_to_check = "Bob"
is_present = name_to_check in scores # No need to loop here

print(f"Total Score: {total_score}")
print(f"Is {name_to_check} in the dictionary? {'Yes' if is_present else 'No'}")
```

```
6
Total Score: 253
Is Bob in the dictionary? Yes
```

Functions

I think the gimmick has worn off and you could see how we could replace functions with a combination of copy pasting code, loops and conditionals. The benefits of functions are however:

- Make code reusable and modular.
- Improve readability and maintainability.

In [ ]:
```python
# def thing_i_want_to_do_many_times_in_different_ways(some_parameter: float, some_input: Dict[str, int]) -> int:
def thing_i_want_to_do_many_times_in_different_ways(
    some_parameter: float,
    some_input: Dict[str, int]
) -> int:
    """
    Performs a specific operation using the given parameter and input dictionary.

    Args:
        some_parameter (float): A multiplier used in the computation.
        some_input (Dict[str, int]): A dictionary with string keys and integer values, used for additional processing.

    Returns:
        int: The result of the operation, computed by multiplying `some_parameter` with the processed value of `some_input`
    """
    a_result = other_stuff(some_input)

    return some_parameter * a_result
```

Where will you see functions

- Besides that we usually ask you to write a function
- Will often import functions from libraries

```
from awesome_library import awesome_analysis

awesome_analysis(your_data, some_settings) # thank you type hints and docstring
```

Functions - Pitfalls

- Not creating your own functions
- Not returning properly
- Wrong documentation/type hints
- Using global variables
- Not testing properly

```
In [ ]: # Create your own functions
        for game in tournament:
    for team in game.teams:
        for player in team:
            process_player(player)

def process_player(player: Player) -> None:
    ...
#-------------------
best_players = []
for game in tournament:
    best_players.append(extract_best_player(game))

def extracted_best_player(game: Game) -> Player:
    '''don't forget'''
    for team in game.teams:
        ...
#-------------------
def prime_artist(artists: List[Artist]) -> List[str]:
    '''don't forget'''
    for artist in artists:
        if is_prime(artist.age):
            ...
```

Printing is not the same as returning!

- Printing writes text to an output
- Returning exits the current function with a certain value
- Notebooks automatically print the last value of a code block
- Only print when you want feedback for yourself or it's explicitly stated in the question

In [14]:
```
# printing returning
        def some_print() -> None:
    print("hello")

def some_return() -> bool:
    return True

variable_print = some_print()
variable_return = some_return()

print("We did the assignments")

4+9 # Just a random expression, will do nothing

print(variable_print)
print(variable_return)

5+2 # this will also be printed because it's the last value
```

```
hello
We did the assignments
None
True
```

Out[14]:

```
7
```

In [ ]:
```python
# You will lose points for not doing type hints and docstrings properly

from typing import List

def calculate_value(base: float, multiplier: float, modifiers: List[float]) -> float:
    """
    Performs a calculation using a base value, a multiplier, and a list of modifiers.

    Args:
        base (float): The starting value for the calculation.
        multiplier (float): A factor to scale the base value.
        modifiers (List[float]):
            A list of additional values to modify the calculation.

    Returns:
        float: The final calculated value.

    Example:
        >>> calculate_value(10, 2, [0.5, 1.5])
        30.0
    """
    # Apply all modifiers to the calculation
    result = base * multiplier * sum(modifiers)
    return result
```

In [15]:
```
# global variables
a = 5
def some_function(b):
    return a+b # Do not do this

# Even Worse
example_input1 = [1,2,3]
example_input2 = True

def another_function(input1, input2):
    if example_input2: # global variable instead of argument!
        return sum(example_input1)
    return 0

another_function([4,5,6], False) # different arguments won't work
```

Out[15]:

```
6
```

In [16]:
```
# Testing
example_input1 = [1,2,3]
example_input2 = True

def another_function(input1, input2):
    if input2:
        return sum(input1)
    return 0

another_function(example_input1, example_input2)
# Add some other tests!
another_function([4,5,6], False)
another_function([4,5,6], True) # To see all results add prints
```

Out[16]:

```
15
```

Collections

```
In [ ]: my_list = [1, 2, 3, 4]

my_tuple = (1, 2, 3, 4)

my_dict = {"key1": "value1", "key2": "value2"}
```

It's extremely useful to structure your data

Let's reverse the question, are there situations where you would need a dictionary?

```
In [ ]: extremely_large_list = list(range(0,1_000_000_000))

if 9999999 in extremely_large_list: # This would have to loop the list
    print('Found it')

extremely_large_dictionary = {str(i): i for i in range(1_000_000_000)}

if "9999999" in extremely_large_dictionary: # This would be much faster
    print('Found it')

# Of course indexing is nicer in dictionaries because it has meaning
extremely_large_dictionary["Some Index"]=12
```

In Short:

- Tuples are immutable, use them if you don't want to change anything.
- Use lists if you might want to add or remove for change things, or if the order of items matters.
- Dictionaries give you indexing with a determined key and fast lookups.

Collections - Pitfalls

- Forgetting mutability
- Nesting
- Not using built in functionality
- Misunderstanding unpacking

In [17]:
```
# mutability
my_list = [1, 2, 3]
my_dictionary = {'a': 5}

def some_function(l, d):
    l.append(4) # will change the list also outside the function
    d['b'] = 6

print(my_list, my_dictionary)
some_function(my_list, my_dictionary)
print(my_list, my_dictionary)
```

```
[1, 2, 3] {'a': 5}
[1, 2, 3, 4] {'a': 5, 'b': 6}
```

In [ ]:
```python
# Nesting

league_data: Dict[str, Dict[str, List[Dict[str, int]]]] = {
        "TeamA": {
            "players": [
                {"name": "Alice", "goals": 5, "assists": 3, "yellow_cards": 2},
                {"name": "Bob", "goals": 2, "assists": 4, "yellow_cards": 1}
            ],
            "matches": [
                {"opponent": "TeamB", "goals_scored": 3, "goals_conceded": 1},
                {"opponent": "TeamC", "goals_scored": 1, "goals_conceded": 1}
            ]
        },
        "TeamB": {
            "players": [
                {"name": "Charlie", "goals": 7, "assists": 2, "yellow_cards": 1},
                {"name": "David", "goals": 4, "assists": 3, "yellow_cards": 0}
            ],
            "matches": [
                {"opponent": "TeamA", "goals_scored": 1, "goals_conceded": 3},
                {"opponent": "TeamC", "goals_scored": 2, "goals_conceded": 2}
            ]
        }
    }
# Looks very scary but with looping and careful use of functions very doable
```

```
In [ ]: # Built in
        # Biggest one
for team in league_data: # This loop is unnecessary for the following conditional
    if "TeamA" in league_data:
        do_something() # This will actually be done for every team


# Unpacking
# tuples and list can be unpacked
coordinates = (4, 6)
x, y = coordinates

# Also in a list
coordinates_list = [(1, 2), (4, 6), (4, 3)]

for x, y in coordinates_list:
    ...

for x, y, z in coordinates_list: # incorrect unpacking gives an error
    ...

# This would work, do you spot the difference?
coordinate1, coordinate2, coordinate3 = coordinates_list
```

Comprehensions

- Just a succinct notation to create collections

```
squares_list = [x**2 for x in range(1, 6)]
square_dict = {x: x**2 for x in range(1, 6)}
```

Not necessary at all, but widely used so you need to be able to understand them.

In [18]:
```python
# Nested comprehensions can become difficult fast

# Create a 3x3 matrix where each element is the product of row and column indices
matrix = [[i * j for i in range(1, 4)] for j in range(1, 4)]
print(matrix)

# Create a 3x3 matrix, but only keep even values
matrix = [[i * j for i in range(1, 4) if i * j % 2 == 0] for j in range(1, 4)]
print(matrix)

# Flatten the list using nested comprehension
flattened = [item for sublist in matrix for item in sublist]
print(flattened)
```

```
[[1, 2, 3], [2, 4, 6], [3, 6, 9]]
[[2], [2, 4, 6], [6]]
[2, 2, 4, 6, 6]
```

Classes

```
In [19]: class Animal:
             def __init__(self, name):
         self.name = name

     def speak(self):
         print(f"{self.name} makes a sound")

class Dog(Animal):
    def bark(self):
        print(f"{self.name} makes a scary sound")

animal = Animal("Generic Animal")
dog = Dog("Buddy")

animal.speak()
dog.speak()
dog.bark()
```

```
Generic Animal makes a sound
Buddy makes a sound
Buddy makes a scary sound
```

Classes are completely unnecessary!

```
In [ ]: Car = {
            "make": "Toyota",
    "model": "Corolla",
    "year": 2020,
    "start": lambda: print("Car is starting..."),
    "stop": lambda: print("Car is stopping...")
}

print(Car["make"])  # Output: Toyota
print(Car["model"]) # Output: Corolla

Car["start"]()  # Output: Car is starting...
Car["stop"]()   # Output: Car is stopping...
```

Just use dictionaries!

Methods are completely unnecessary!

```
In [ ]: class Circle:
            def __init__(self, radius: float) -> None:
        self.radius = radius

    def area(self) -> float:
        return 3.14 * self.radius**2

def area(circle: Circle) -> float: # exactly the same as the method
    return 3.14 * circle.radius**2

circle = Circle(5)
print(circle.area())  # Output: 78.5
print(area(circle))  # Output: 78.5
```

Just use regular functions!

Inheritance is completely unnecessary!

```
In [ ]: class Animal:
            def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound")

class Dog:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound")

    def bark(self):
        print(f"{self.name} makes a scary sound")
```

Just copy pasta!

I hope it's clear that this is a joke, but why would you use classes?

- Encapsulation of data and abstraction
- It can be an intuitive way to think about problems
- It is what the world uses

```
from openai import OpenAI # a class to interact with the API
```

Methods especially make sense when you think about state.

```
In [ ]: class Circle:
            def __init__(self, radius, x=0, y=0):
        self.radius = radius
        self.x = x
        self.y = y

    def move(self, dx, dy):
        self.x += dx
        self.y += dy

def move(circle: Circle, dx, dy):
    circle.x += dx
    circle.y += dy

circle = Circle(radius=5, x=10, y=15)

print(f"Initial position: ({circle.x}, {circle.y})")
circle.move(5, -3) # makes it more clear that the circle is being moved
move(circle, 1, 2) # here the circle seems more of an input than the object being affected
print(f"New position: ({circle.x}, {circle.y})")
```

In [21]:
```python
# Cool people use method chaining
class Circle:
    def __init__(self, radius, x=0, y=0):
        self.radius = radius
        self.x = x
        self.y = y

    def move(self, dx, dy) -> 'Circle': # necessary if you refer to own type
        self.x += dx
        self.y += dy
        return self # now we can chain the method

circle = Circle(radius=5, x=10, y=15)

print(f"Initial position: ({circle.x}, {circle.y})")
circle.move(5, -3).move(10, 5).move(-5, -2)
print(f"New position: ({circle.x}, {circle.y})")
```

```
Initial position: (10, 15)
New position: (20, 15)
```

Besides preventing code duplication inheritance is also common when using external libraries.

```
In [ ]: # Real reinforcement learning neural network example
        import torch
import torch.nn as nn

class DQN(nn.Module):

    def __init__(self, n_observations, n_actions):
        super(DQN, self).__init__()
        self.layer1 = nn.Linear(n_observations, 128)
        self.layer2 = nn.Linear(128, 128)
        self.layer3 = nn.Linear(128, n_actions)
```

Classes - Pitfalls

- Forgetting `self`
- Class versus instance variables
- `__init__` mistakes
- Inheritance and `super()`

```
In [ ]: class Car:
            def wrong_method():  # Missing self!
        print("This will cause an error")

    def correct_method(self):  # Correct
        print("This works")

class Student:
    # Class variable - shared by all instances
    grades = []

    def add_grade(self, grade):
        self.grades.append(grade)  # This affects ALL instances!

# Better approach:
class Student:
    def __init__(self):
        # Instance variable - unique to each instance
        self.grades = []

    def print_the_grades(self):
        print(grades) # Forgetting self will still cause problems here
```

In [22]:
```
class Person:
        def set_name(self, name):
        self.name = name

    def greet(self):
        # This will fail if set_name wasn't called first
        print(f"Hello, {self.name}")

# Better approach:
class Person:
    def __init__(self, name):
        self.name = name  # Initialize with object

class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        # Forgetting to initialize the parent class
        self.breed = breed  # name won't be set!

# Correct approach:
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
```

Exceptions

```
In [23]: def function_where_something_can_go_wrong():
             if some_condition:
         raise Exception("Things went wrong")

try:
    function_where_something_can_go_wrong()
except Exception as e:
    print(e) # Will actually print that some_condition is not defined
```

```
name 'some_condition' is not defined
```

Things can and will go wrong, so we need to deal with those errors/exceptions.

```
In [24]: # Exceptions are different than returning
         # Exceptions will propagate until handled

def a():
    b()

def b():
    c()
    print('this will not be printed')

def c():
    raise Exception

a() # unhandled exceptions will give you this scary traceback, just like any error
```

```
---------------------------------------------------------------------------
Exception                                 Traceback (most recent call last)
Cell In [24], line 14
     11 def c():
     12     raise Exception
---> 14 a()

Cell In [24], line 5, in a()
      4 def a():
----> 5     b()

Cell In [24], line 8, in b()
      7 def b():
----> 8     c()
      9     print('this will not be printed')

Cell In [24], line 12, in c()
     11 def c():
---> 12     raise Exception
```

```
Exception:
```

In [26]:
```python
# You should be more specific than just a generic Exception

class NegativeValueError(Exception):
    """Custom exception for handling negative values."""
    def __init__(self, value):
        super().__init__(f"Invalid value: {value}. Value must be non-negative.")
        self.value = value

def calculate_square_root(value):
    if value < 0:
        raise NegativeValueError(value)  # Raise the custom exception
    return value ** 0.5

try:
    result = calculate_square_root(-9)
    print(f"The square root is {result}")
except NegativeValueError as e:
    print(e)  # Output: Invalid value: -9. Value must be non-negative.
```

```
Invalid value: -9. Value must be non-negative.
```